Ajaia LLC: AI-Native Full Stack Developer Assignment

Docs with Friends - Architecture Note

Overview:

Docs with Friends is a small, single-server web application for creating, editing, importing, sharing, and downloading documents. It uses a vanilla JavaScript frontend, a Node.js and Express backend, SQLite for structured data, and local files for uploaded attachments. The browser and server communicate through JSON API endpoints.

Main parts:

The browser interface is served from the public directory. index.html provides the page shell, styles.css controls presentation, and app.js renders the screens, manages navigation and editor state, and sends requests to the API. The frontend is a single-page interface using URL hash routes; it does not require a frontend framework or build step.

server.js creates the SQLite database and Express application, then listens on the configured PORT (3000 by default). src/app.js configures JSON parsing, authentication and document routes, serves the static frontend, and handles errors. Route handlers are separated into auth.js and documents.js, while require-user.js and auth-cookie.js handle authentication checks and signed cookies.

The /auth endpoints list the available seeded users and log users in or out. The /api/documents endpoints create, list, read, update, import, share, and delete documents, and manage attachment uploads and downloads. The documents router checks whether a user owns or has been given access to a document before allowing protected operations. Only the owner can rename or delete a document or change its sharing settings; collaborators can edit its content.

Data and file storage:

SQLite stores users, documents, sharing relationships, and attachment metadata. Foreign keys connect documents to owners and attachments and shares to their documents. The database is created at data/app.db by default. Attachment bytes are stored separately under data/uploads, with a generated storage name; their original filename and other metadata remain in SQLite. A persistent volume mounted at /app/data keeps both database records and uploaded files when the Docker container is replaced.

Security and boundaries:

Login uses an HMAC-signed, HttpOnly cookie. The cookie secret is configured with DOCS_COOKIE_SECRET for deployments. Document access is checked on the server for each protected request rather than trusted from the front-end state.

Document HTML is sanitized on the server before it is saved, and imported DOCX content is sanitized after conversion. Uploads have size limits (10 mb), and imports are restricted to TXT, Markdown, and DOCX.

What is incomplete:

SQLite and local disk keep setup and deployment simple, but this design is intended for a single basic application instance with a persistent data volume; it can be replaced in a stretch goal. Multiple instances would need a shared database and file storage. The frontend uses browser-native editing rather than a larger editor framework, and the seeded-user login is a demo approach rather than production authentication. These choices keep the application small and easy to run, while leaving room to add stretch goals: managed authentication, object storage, or a more advanced editor.